

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №6

Специальность ПО-13

Выполнил:
М.А. Романюк
студент группы ПО-13
Проверил:
А.А.Крощенко
доц. кафедры ИИТ,
14.05.2026

Брест 2026

Цель работы: освоить приемы тестирования кода на примере использования пакета `pytest`

Общее задание:

Задание 1:

Написание тестов для мини-библиотеки покупок (`shopping.py`)

1. Создайте файл `test_cart.py`. Реализуйте следующие тесты:

- Проверка добавления товара: после `add_item("Apple", 10.0)` в корзине должен быть один элемент.
- Проверка выброса ошибки при отрицательной цене.
- Проверка вычисления общей стоимости (`total()`).

2. Протестируйте метод `apply_discount` с разными значениями скидки:

- 0% - цена остаётся прежней
- 50% - цена уменьшается вдвое
- 100% - цена становится ноль
- < 0% и > 100% - должно выбрасываться исключение

Используйте `@pytest.mark.parametrize`

3. Создайте фикстуру `empty_cart`, которая возвращает пустой экземпляр `Cart`

```
@pytest.fixture
def empty_cart():
    return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.

4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:

```
import requests
def log_purchase(item):
    requests.post("https://example.com/log", json=item)
```

- Замокните `requests.post`, чтобы не было реального HTTP-запроса
- Убедитесь, что он вызывается с корректными данными

5. Добавьте поддержку купонов:

```
def apply_coupon(cart, coupon_code):
    coupons = {"SAVE10": 10, "HALF": 50}
    if coupon_code in coupons:
        cart.apply_discount(coupons[coupon_code])
    else:
        raise ValueError("Invalid coupon")
```

- Напишите тесты на `apply_coupon`

- Замокайте словарь coupons с помощью monkeypatch или patch.dict

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случаи, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

Вариант 9:

Реализуйте метод `int hammingDistance(String str1, String str2)`, определяющий расстояние Хэминга для двух строк. Дистанция Хэминга – это число позиций, в которых соответствующие символы двух слов одинаковой длины различны.

Спецификация метода:

```
hammingDistance (None, None) = TypeError
hammingDistance (None, *) = -1
hammingDistance (* , None) = -1
hammingDistance ("abc", " abcd ") = ValueError
hammingDistance ("","") = 0
hammingDistance (" father ", " father ") = 0
hammingDistance ("pip", " pop ") = 1
hammingDistance (" abcd ", " abab ") = 2
hammingDistance (" hello ", " hallo ") = 1
hammingDistance (" abcd ", " efgi ") = 4
```

Код программы:

Задание 1:

shopping.py

```
import requests

coupons = {"SAVE10": 10, "HALF": 50}

class Cart:
    def __init__(self):
```

```

        self.items = []

    def add_item(self, name: str, price: float):
        if price < 0:
            raise ValueError("Price cannot be negative")
        self.items.append({"name": name, "price": price})

    def total(self):
        return sum(item["price"] for item in self.items)

    def apply_discount(self, percent: float):
        if percent < 0 or percent > 100:
            raise ValueError("Discount must be between 0 and 100")
        for item in self.items:
            item["price"] *= (1 - percent / 100)

    def log_purchase(item):
        requests.post("https://example.com/log", json=item, timeout=5)

    def apply_coupon(cart, coupon_code):
        if coupon_code in coupons:
            cart.apply_discount(coupons[coupon_code])
        else:
            raise ValueError("Invalid coupon")

```

test_cart.py

```

# pylint: disable=redefined-outer-name
from unittest.mock import patch
import pytest
from shopping import Cart, log_purchase, apply_coupon

@pytest.fixture
def empty_cart():
    return Cart()

def test_add_item(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1
    assert empty_cart.items[0]["name"] == "Apple"
    assert empty_cart.items[0]["price"] == 10.0

def test_add_item_negative_price(empty_cart):
    with pytest.raises(ValueError):
        empty_cart.add_item("Banana", -5.0)

def test_total(empty_cart):

```

```

empty_cart.add_item("Apple", 10.0)
empty_cart.add_item("Banana", 5.0)
assert empty_cart.total() == 15.0

@pytest.mark.parametrize("discount, expected_total", [
    (0, 15.0),
    (50, 7.5),
    (100, 0.0),
])
def test_apply_discount(discount, expected_total, empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 5.0)
    empty_cart.apply_discount(discount)
    assert empty_cart.total() == expected_total

@pytest.mark.parametrize("discount", [-10, 150])
def test_apply_discount_invalid(discount, empty_cart):
    empty_cart.add_item("Apple", 10.0)
    with pytest.raises(ValueError):
        empty_cart.apply_discount(discount)

def test_log_purchase():
    item = {"name": "Apple", "price": 10.0}
    with patch("shopping.requests.post") as mock_post:
        log_purchase(item)
        mock_post.assert_called_once_with(
            "https://example.com/log", json=item, timeout=5
        )

def test_apply_coupon_valid(empty_cart, monkeypatch):
    empty_cart.add_item("Item", 100.0)
    monkeypatch.setitem(__import__('shopping').coupons, "TEST10", 10)

    apply_coupon(empty_cart, "TEST10")

    assert empty_cart.total() == 90.0

def test_apply_coupon_invalid(empty_cart, monkeypatch):
    empty_cart.add_item("Item", 100.0)
    monkeypatch.setitem(__import__('shopping').coupons, "TEST10", 10)

    with pytest.raises(ValueError):
        apply_coupon(empty_cart, "BADCODE")\

```

11 passed in 0.29s

Задание 2:

task.py

```
def zadanie1():
    numbers = input("Введите числа: ").split()
    a = b = c = 0
    for n in numbers:
        if int(n) < 10 and int(n) > -10:
            a += 1
        elif int(n) < 100 and int(n) > -100:
            b += 1
        elif int(n) < 1000 and int(n) > -1000:
            c += 1
    print("Однозначных:", a)
    print("Двузначных:", b)
    print("Трёхзначных:", c)
    return a, b, c
```

```
def zadanie2():
    results = []
    while True:
        text = input("Input: ")
        if text == "":
            break
        number = int(text)
        binary = bin(number)
        count = 0
        for symbol in binary:
            if symbol == "1":
                count = count + 1
        print("Output:", count)
        print()
        results.append(count)
    return results
```

tests.py

```
"""Tests for zadanie1 and zadanie2."""
import pytest
from task import zadanie1, zadanie2

class TestZadanie1:
    """Tests for zadanie1 function."""

    def test_trivialnye_odnoznachnye(self, monkeypatch):
        """Test single digit numbers."""
```

```

monkeypatch.setattr('builtins.input', lambda _: "1 2 3")
monkeypatch.setattr('builtins.print', lambda *x: None)
a, b, c = zadanie1()
assert a == 3
assert b == 0
assert c == 0

def test_triwiialnye_dvuznachnye(self, monkeypatch):
    """Test double digit numbers."""
    monkeypatch.setattr('builtins.input', lambda _: "10 20 30")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    a, b, c = zadanie1()
    assert a == 0
    assert b == 3
    assert c == 0

def test_triwiialnye_trehznachnye(self, monkeypatch):
    """Test three digit numbers."""
    monkeypatch.setattr('builtins.input', lambda _: "100 200 300")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    a, b, c = zadanie1()
    assert a == 0
    assert b == 0
    assert c == 3

def test_granichnye_9_i_10(self, monkeypatch):
    """Test boundary 9 and 10."""
    monkeypatch.setattr('builtins.input', lambda _: "9 10")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    a, b, c = zadanie1()
    assert a == 1
    assert b == 1
    assert c == 0

def test_granichnye_minus9_i_minus10(self, monkeypatch):
    """Test boundary -9 and -10."""
    monkeypatch.setattr('builtins.input', lambda _: "-9 -10")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    a, b, c = zadanie1()
    assert a == 1
    assert b == 1
    assert c == 0

def test_granichnye_99_i_100(self, monkeypatch):
    """Test boundary 99 and 100."""
    monkeypatch.setattr('builtins.input', lambda _: "99 100")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    a, b, c = zadanie1()

```

```
assert a == 0
assert b == 1
assert c == 1
```

```
def test_granichnye_minus99_i_minus100(self, monkeypatch):
    """Test boundary -99 and -100."""
    monkeypatch.setattr('builtins.input', lambda _: "-99 -100")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    a, b, c = zadanie1()
    assert a == 0
    assert b == 1
    assert c == 1
```

```
def test_nol(self, monkeypatch):
    """Test zero."""
    monkeypatch.setattr('builtins.input', lambda _: "0")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    a, b, c = zadanie1()
    assert a == 1
    assert b == 0
    assert c == 0
```

```
def test_otricatelnye_chisla(self, monkeypatch):
    """Test negative numbers."""
    monkeypatch.setattr('builtins.input', lambda _: "-5 -50 -500")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    a, b, c = zadanie1()
    assert a == 1
    assert b == 1
    assert c == 1
```

```
def test_smeshannye_chisla(self, monkeypatch):
    """Test mixed numbers."""
    monkeypatch.setattr('builtins.input', lambda _: "5 -5 10 -10 100 -100")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    a, b, c = zadanie1()
    assert a == 2
    assert b == 2
    assert c == 2
```

```
def test_chisla_vne_diapazona(self, monkeypatch):
    """Test numbers outside range."""
    monkeypatch.setattr('builtins.input', lambda _: "1000 10000 -1000")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    a, b, c = zadanie1()
    assert a == 0
    assert b == 0
    assert c == 0
```

```

def test_pustoy_vvod(self, monkeypatch):
    """Test empty input."""
    monkeypatch.setattr('builtins.input', lambda _: "")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    a, b, c = zadanie1()
    assert a == 0
    assert b == 0
    assert c == 0

def test_isklyuchitelnaya_situaciya_ne_chislo(self, monkeypatch):
    """Test exception on non-number."""
    monkeypatch.setattr('builtins.input', lambda _: "abc 10")
    monkeypatch.setattr('builtins.print', lambda *x: None)
    with pytest.raises(ValueError):
        zadanie1()

class TestZadanie2:
    """Tests for zadanie2 function."""

    def test_trivialnye_5(self, monkeypatch):
        """Test number 5."""
        inputs = iter(["5", ""])
        monkeypatch.setattr('builtins.input', lambda _: next(inputs))
        monkeypatch.setattr('builtins.print', lambda *x: None)
        results = zadanie2()
        assert results == [2]

    def test_trivialnye_0(self, monkeypatch):
        """Test number 0."""
        inputs = iter(["0", ""])
        monkeypatch.setattr('builtins.input', lambda _: next(inputs))
        monkeypatch.setattr('builtins.print', lambda *x: None)
        results = zadanie2()
        assert results == [0]

    def test_trivialnye_1(self, monkeypatch):
        """Test number 1."""
        inputs = iter(["1", ""])
        monkeypatch.setattr('builtins.input', lambda _: next(inputs))
        monkeypatch.setattr('builtins.print', lambda *x: None)
        results = zadanie2()
        assert results == [1]

    def test_granichnye_stepeni_dvoyki(self, monkeypatch):
        """Test powers of two."""
        inputs = iter(["2", "4", "8", "16", "32", ""])

```

```

monkeypatch.setattr('builtins.input', lambda _: next(inputs))
monkeypatch.setattr('builtins.print', lambda *x: None)
results = zadanie2()
assert results == [1, 1, 1, 1, 1]

def test_granichnye_vse_edinicy(self, monkeypatch):
    """Test all ones."""
    inputs = iter(["7", "15", "31", "63", "127", "255", ""])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))
    monkeypatch.setattr('builtins.print', lambda *x: None)
    results = zadanie2()
    assert results == [3, 4, 5, 6, 7, 8]

def test_otricatelnye_chisla(self, monkeypatch):
    """Test negative numbers."""
    inputs = iter(["-1", "-2", "-3", "-5", ""])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))
    monkeypatch.setattr('builtins.print', lambda *x: None)
    results = zadanie2()
    assert results == [1, 1, 2, 2]

def test_bolshie_chisla(self, monkeypatch):
    """Test large numbers."""
    inputs = iter(["1023", "1024", ""])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))
    monkeypatch.setattr('builtins.print', lambda *x: None)
    results = zadanie2()
    assert results == [10, 1]

def test_neskolko_chisel_podryad(self, monkeypatch):
    """Test multiple numbers in a row."""
    inputs = iter(["5", "10", "15", "0", "7", ""])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))
    monkeypatch.setattr('builtins.print', lambda *x: None)
    results = zadanie2()
    assert results == [2, 2, 4, 0, 3]

def test_pustoy_vvod_srazu(self, monkeypatch):
    """Test empty input immediately."""
    inputs = iter([""])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))
    monkeypatch.setattr('builtins.print', lambda *x: None)
    results = zadanie2()
    assert not results

def test_isklyuchitelnaya_situaciya_ne_chislo(self, monkeypatch):
    """Test exception on non-number."""
    inputs = iter(["abc", ""])

```

```

monkeypatch.setattr('builtins.input', lambda _: next(inputs))
monkeypatch.setattr('builtins.print', lambda *x: None)
with pytest.raises(ValueError):
    zadanie2()

def test_isklyuchitelnaya_situaciya_pustaya_stroka_v_seredine(self, monkeypatch):
    """Test empty string in middle."""
    inputs = iter(["5", "", "10"])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))
    monkeypatch.setattr('builtins.print', lambda *x: None)
    results = zadanie2()
    assert results == [2]

def test_isklyuchitelnaya_situaciya_float(self, monkeypatch):
    """Test exception on float."""
    inputs = iter(["3.14", ""])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))
    monkeypatch.setattr('builtins.print', lambda *x: None)
    with pytest.raises(ValueError):
        zadanie2()

```

```

= 25 passed in 0.07s =

```

Задание 3:

task3.py

```

def hamming_distance(str1, str2):
    if str1 is None and str2 is None:
        raise TypeError
    if str1 is None or str2 is None:
        return -1
    if not isinstance(str1, str) or not isinstance(str2, str):
        return -1
    if len(str1) != len(str2):
        raise ValueError
    distance = 0
    for i, char in enumerate(str1):
        if char != str2[i]:
            distance += 1
    return distance

```

test3.py

```

import pytest

```

```
from task3 import hamming_distance
```

```
class TestHammingDistance:
    def test_both_none(self):
        with pytest.raises(TypeError):
            hamming_distance(None, None)

    def test_first_none(self):
        assert hamming_distance(None, "abc") == -1

    def test_second_none(self):
        assert hamming_distance("abc", None) == -1

    def test_not_string_first(self):
        assert hamming_distance(123, "abc") == -1

    def test_not_string_second(self):
        assert hamming_distance("abc", 123) == -1

    def test_different_length(self):
        with pytest.raises(ValueError):
            hamming_distance("abc", "abcd")

    def test_both_empty(self):
        assert hamming_distance("", "") == 0

    def test_same_string(self):
        assert hamming_distance("father", "father") == 0

    def test_pip_pop(self):
        assert hamming_distance("pip", "pop") == 1

    def test_abcd_abab(self):
        assert hamming_distance("abcd", "abab") == 2

    def test_hello_hallo(self):
        assert hamming_distance("hello", "hallo") == 1

    def test_abcd_efgi(self):
        assert hamming_distance("abcd", "efgi") == 4

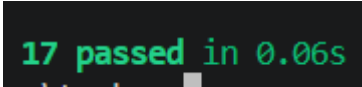
    def test_case_sensitive(self):
        assert hamming_distance("Hello", "hello") == 1

    def test_numbers_as_strings(self):
        assert hamming_distance("123", "124") == 1
```

```
def test_long_strings(self):
    assert hamming_distance("abcdefgh", "abcdefgh") == 0
    assert hamming_distance("abcdefgh", "abcdefghi") == 1

def test_empty_vs_non_empty(self):
    with pytest.raises(ValueError):
        hamming_distance("", "a")

def test_unicode_characters(self):
    assert hamming_distance("Привет", "Привет") == 0
    assert hamming_distance("Привет", "Привет") == 1
```



17 passed in 0.06s

Вывод: освоил приемы тестирования кода на примере использования пакета pytest